CONTEXT-AWARE ACTIVITY RECOGNITION: AN EXPLAINABLE WEARABLE IOT SYSTEM FOR REAL TIME FALL DETECTION IN ELDERLY PEOPLE

Mallawa Thanthrige Kavindya Chathuri
IT22548078
R26-IT-051

Bachelor of Science (Hons) in Information Technology Specialize in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy (see Appendix B). I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Mallawa Thanthrige Kavindya Chathuri

Student ID: IT22548078

Signature: _____ Date: ____2026/10/11_____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Mr.Uditha Dharmakeerthi		
Co-supervisor: Mr.Kanishka Yapa		

ABSTRACT

Falls are a leading cause of injury and death among older adults, and wearable detectors are often distrusted because they raise false alarms and do not explain their decisions. This thesis presents the Context-Aware Activity Recognition component of the AegisCare elderly-safety platform, covering the complete chain from sensor to caregiver, with hardware, firmware, networking, machine learning, decision logic and interface all developed individually. A wearable node, a BMI160 six-axis inertial sensor on a Wemos D1 Mini (ESP8266), is calibrated, filtered with an exponential moving average and streams data at about 10 Hz over MQTT, secured with TLS, to a Flask service that exposes a REST interface. The service groups samples into 20-sample windows, extracts 52 statistical features and classifies them with a Random Forest into six motion classes: standing, walking and falls in four directions. A stateful Fall Engine confirms a fall only when three gates agree: the classifier reports a fall with at least 60% confidence, the measured trunk tilt reaches 30°, and the body then stays tilted and still. A separate posture-decision layer grades movement on six levels. The engine also produces a 0–100 fall-risk score and a plain-language explanation of each decision, and a React dashboard with eight sections shows the results live. The model was developed on 14,209 samples (1,077 windows) from 10 participants. On people it had never seen, the Random Forest reached 91.97 ± 5.37% accuracy in leave-one-subject-out cross-validation (range 81.65–99.39%) and 92.12% (weighted F1 92.07%) on two held-out participants; right falls and standing still were the least reliably recognised classes. The component was checked with API, engine and dashboard test cases. Falls were simulated by healthy volunteers, so validation with older adults remains future work. The contribution is a transparent, explainable, end-to-end fall-confirmation design evaluated on unseen people.

Keywords: fall detection, human activity recognition, inertial measurement unit, Random Forest, explainable decision support

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, Mr. Uditha Dharmakeerthi and my co-supervisor, Mr. Kanishka Yapa, for their invaluable guidance, constructive criticism and continuous encouragement throughout this research. Their patience in reviewing my work, their insightful questions and their willingness to give time whenever I needed direction shaped both the research questions and the way the results are reported in this thesis. The standards they set challenged me to test my ideas honestly and to report the limitations of my work as clearly as its achievements.

I am deeply grateful to the members of the AegisCare research group, whose work on the cardiovascular, environmental-risk and smart-alert components made it possible to test my component as part of a complete elderly-safety platform. The discussions, shared problem-solving and teamwork within the group helped me understand how an individual component must behave when it is combined with the work of others.

I sincerely thank the ten participants who volunteered their time to take part in the data collection. Wearing the sensor and repeating standing, walking and simulated falls many times was not easy, and the dataset on which this research is built would not exist without their patience and cooperation.

I also thank the academic and technical staff of the Department of Information Technology, Sri Lanka Institute of Information Technology, for the laboratory access, equipment and support that made the hardware development, data collection and testing possible. I am grateful to the lecturers who taught me the foundations of programming, machine learning, networking and research methods that this work builds on, and to the examiners and panel members whose feedback during the progress presentations helped me improve the research.

I extend my appreciation to my friends and fellow students, who offered encouragement, helped with testing and reminded me to keep going during the difficult stages of this project.

Finally and most importantly, I thank my family for their unconditional love, patience and support. Their belief in me and their sacrifices throughout my studies, gave me the strength to complete this thesis.

TABLE OF CONTENTS

Page

DECLARATION AND SUPERVISOR CERTIFICATION....ii
ABSTRACT....iii
ACKNOWLEDGEMENTS....iv
TABLE OF CONTENTS....v
LIST OF FIGURES....viii
LIST OF TABLES....ix
LIST OF ABBREVIATIONS....x
LIST OF APPENDICES....xi
1 INTRODUCTION....1
1.1 Overall project context....1
1.2 Individual component context....1
1.3 Problem statement....3
1.4 Research gap....3
1.5 Aim and objectives....4
1.6 Research questions or hypotheses....5
1.7 Scope limitations and assumptions....5
1.8 Individual contribution....6
1.9 Thesis structure....6
2 LITERATURE REVIEW AND RELATED WORK....7
2.1 Review method and scope....7
2.2 Domain and theoretical foundations....7
2.2.1 Falls in older adults....7
2.2.2 Inertial sensing....7
2.2.3 Windowed activity recognition....8
2.2.4 Random Forest....8
2.2.5 Explainability....8
2.3 Related research....9
2.3.1 Surveys of fall detection....9
2.3.2 Threshold-based detection....9
2.3.3 Machine-learning detection....9
2.3.4 Evaluation validity....10
2.4 Existing systems products and techniques....10
2.5 Gap synthesis and conceptual framework....11
3 RESEARCH METHODOLOGY....13
3.1 Research design....13
3.2 Data participants or study materials....13

3.2.1 Hardware and acquisition....13
3.2.2 Dataset....14
3.2.3 Software and libraries....15
3.3 Experimental or study procedure....15
3.3.1 Windowing and feature extraction....15
3.3.2 Classifiers....15
3.3.3 Evaluation protocols....16
3.3.4 Threshold calibration for the confirmation engine....16
3.3.5 Scripted scenarios for the full pipeline....16
3.4 Measures and evaluation metrics....17
3.5 Data analysis....17
3.6 Ethics privacy security and responsible AI....17
3.7 Reproducibility and research artefacts....18
4 PRODUCT AND SOLUTION DESIGN....19
4.1 Requirements....19
4.2 Component architecture within the complete solution....19
4.3 Design alternatives and decisions....20
4.4 Detailed design....21
4.4.1 Sensor node and message format....21
4.4.2 Feature extraction and classifier....21
4.4.3 Fall Engine: three-gate confirmation....21
4.4.4 Fall-risk score....23
4.4.5 Explanations....23
4.4.6 Posture-decision logic in the dashboard....24
4.4.7 Dashboard....24
4.5 Standards and professional practice....25
5 IMPLEMENTATION INTEGRATION AND TESTING....27
5.1 Development environment and technology stack....27
5.2 Component implementation....27
5.2.1 Firmware....27
5.2.2 Service....27
5.2.3 Fall Engine....28
5.2.4 Dashboard....28
5.3 Integration with the other components....28
5.4 Verification and software testing....29
5.5 Deployment and operation....30
6 RESULTS AND VALIDATION....31
6.1 Research results....31
6.1.1 Classifier accuracy under two evaluation protocols....31

6.1.2 Per-class performance and confusions....32
6.1.3 Feature importance....33
6.2 Product and technical results....34
6.2.1 Tilt angle by class and the choice of the 30° gate....34
6.2.2 Full-pipeline scenarios....35
6.3 User or stakeholder validation....36
6.4 Integration results....37
6.5 Summary of findings....38
7 DISCUSSION....39
7.1 Interpretation of findings....39
7.2 Comparison with literature baselines and existing solutions....40
7.3 Research contribution....40
7.4 Product contribution....41
7.5 Limitations and threats to validity....41
7.6 Implications....42
8 COMMERCIALISATION AND SUSTAINABILITY....43
8.1 Value proposition and target users....43
8.2 Market and competitive position....43
8.3 Business or sustainability model....43
8.4 Scalability intellectual property and compliance....43
8.5 Contribution to the Sustainable Development Goals....44
9 CONCLUSION AND FUTURE DIRECTIONS....45
9.1 Conclusion....45
9.2 Achievement of objectives....45
9.3 Individual research and product contributions....46
9.4 Future directions....46
REFERENCES....47
GLOSSARY....49
APPENDICES....50
Appendix A Extended results and test evidence....51
Appendix B Academic integrity and AI use disclosure....53

LIST OF FIGURES

Page

Figure 1.1 IoT hardware and AI system overview of the Context-Aware Activity Recognition component....2
Figure 2.1 Conceptual framework: three gates must agree before a fall is confirmed....12

Figure 4.1 Fall Engine decision flow: the three gates and the outcome of each failure.....23
Figure 4.2 Posture-decision section of the dashboard showing a slight backward tilt of 17° ("not a fall").....25
Figure 6.1 Accuracy of the three models under the random split and under session-grouped cross-validation.....32
Figure 6.2 Confusion matrix of the Random Forest under session-grouped cross-validation (counts of windows).....33
Figure 6.3 Ten most important features of the Random Forest (impurity-based importance).....33
Figure 6.4 Distribution of the peak tilt per session by class with the 30° gate.....35

LIST OF TABLES

Page

Table 1.1 Research questions and hypotheses.....5
Table 1.2 Individual contributions and the artefacts that evidence them.....6
Table 2.1 Qualitative comparison of fall-detection approaches.....10
Table 2.2 Identified gaps and how the component addresses them.....11
Table 3.1 Stages of the study and the research question each stage answers.....13
Table 3.2 Composition of the data set.....14
Table 3.3 Feature set computed for every window.....15
Table 4.1 Functional and non-functional requirements.....19
Table 4.2 REST interface of the activity-recognition service (port 5050).....20
Table 4.3 Design alternatives and decisions.....20
Table 4.4 Thresholds of the Fall Engine.....22
Table 4.5 Posture levels of the dashboard decision logic.....24
Table 4.6 Sections of the live dashboard.....24 Table 5.1 Technology stack of the component.....27
Table 5.2 Test cases.....29
Table 6.1 Results of the random 80/20 split (protocol P1).....31
Table 6.2 Results of session-grouped 5-fold cross-validation (protocol P2), mean ± standard deviation.....31
Table 6.3 Per-class results of the Random Forest (out-of-fold predictions).....32
Table 6.4 Peak tilt per session (degrees).....34
Table 6.5 Session-level results of the full pipeline (classifier, 60% rule, Fall Engine).....35
Table 6.6 Results of the dashboard posture engine on the recorded sessions.....37
Table 6.7 Summary of findings by research question.....38
Table 7.1 Limitations and threats to validity.....41
Table 9.1 Achievement of the objectives.....45
Table A.1 Confusion matrix of the Random Forest (rows: actual; columns: predicted; out-of-fold windows).....51
Table A.2 API test log (Flask test client, MQTT stubbed).....51
Table B.1 Use of generative AI tools.....53

LIST OF ABBREVIATIONS

Abbreviation	Description
ADL	Activities of Daily Living
API	Application Programming Interface
CV	Cross-Validation
DT	Decision Tree
EMA	Exponential Moving Average
ESP8266	Wi-Fi microcontroller used on the Wemos D1 Mini board
F1	Harmonic mean of precision and recall
HAR	Human Activity Recognition
I²C	Inter-Integrated Circuit (serial bus)
IMU	Inertial Measurement Unit
IoT	Internet of Things
JSON	JavaScript Object Notation
KNN	k-Nearest Neighbours
LSB	Least Significant Bit
ML	Machine Learning
MQTT	Message Queuing Telemetry Transport
PDPA	Personal Data Protection Act (Sri Lanka)
REST	Representational State Transfer
RF	Random Forest
SDG	Sustainable Development Goal
SGKF	Stratified Group K-Fold
TLS	Transport Layer Security
WHO	World Health Organization
XAI	Explainable Artificial Intelligence

LIST OF APPENDICES

Page

Appendix A Extended results and test evidence.....51
Appendix B Academic integrity and AI use disclosure.....53

1 INTRODUCTION

1.1 Overall project context

Population ageing is changing how societies organize care. As people live longer, more older adults spend part of their day alone and a growing share of them live with at least one chronic condition. Falls are among the most common and most preventable consequences of this situation. The World Health Organization reports that falls are the second leading cause of unintentional injury deaths worldwide, that adults older than 60 years suffer the greatest number of fatal falls and that an estimated 684 000 people die from falls each year [1]. Population studies of community-dwelling older adults have long reported that roughly one in three people aged 65 and over falls at least once a year [2]. The harm caused by a fall depends not only on the fall itself but on how quickly help arrives, because a person who cannot get up may lie on the floor for hours.

AegisCare is a research platform that addresses this problem as a family of cooperating components rather than as a single sensor. Its React web application gives caregivers one place to monitor an older person and its back end is split into independent services for activity recognition, cardiovascular monitoring, environmental-risk assessment, and smart alerts. Each service is owned by a different member of the research group and is exposed to the web application through its own REST interface. The platform therefore needs every component to behave predictably when it is combined with the others: it must publish its output in a stable format, it must stay responsive when another service is slow, and it must be understandable enough for a caregiver to trust.

This thesis reports the individual work on one of these components, **Context-Aware Activity Recognition**, which is the part of AegisCare that watches how the older person moves and decides whether a fall has happened.

1.2 Individual component context

The Context-Aware Activity Recognition component is a complete vertical slice: a wearable sensor node, a message broker connection, a machine-learning service, a physics-based confirmation engine and a live web dashboard. A Bosch BMI160 six axis inertial measurement unit (IMU) is attached to a Wemos D1 Mini (ESP8266) board. The firmware calibrates the sensor, smooths the readings with an exponential moving average, and publishes a JSON message ten times per second over the Message

Queuing Telemetry Transport (MQTT) protocol [3]. A Flask service subscribes to these messages, groups them into windows of 20 samples, extracts 52 statistical features from each window and passes them to a Random Forest classifier [4] that recognizes six motion classes: standing still, normal walking and falls to the forward, backward, left and right. A stateful Fall Engine then combines the classifier output with the measured body tilt and with what happens after the movement and the dashboard presents the result as a posture level, a fall-risk score and a plain-language explanation.

Figure 1.1 places the component in the complete AegisCare solution. The component is the only one in the platform that depends on a custom hardware device, which is why hardware, firmware, network, machine learning, back-end logic and user interface are all part of this thesis.

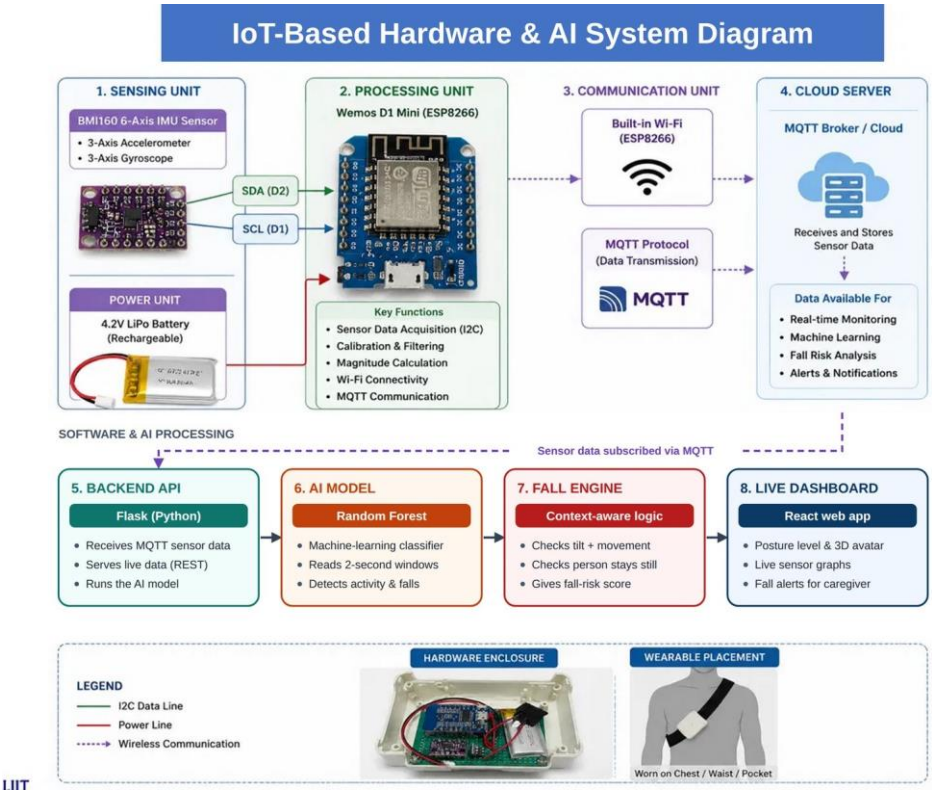


Figure 1.1 IoT hardware and AI system overview of the Context-Aware Activity Recognition component *Source: Author.*

1.3 Problem statement

Wearable fall detectors are widely available, yet they are not trusted by the people who would benefit most from them. Two failures are responsible. The first is the **false alarm**: a movement such as bending to tie a shoelace, sitting down quickly, or stumbling and recovering produces a sensor signal that resembles a fall, so the device

raises an alert that the caregiver then learns to ignore. The second is the wrong decision: when a classifier reports a fall, neither the older person nor the caregiver is told why, which makes it impossible to judge whether the alert is credible or to improve the system when it is wrong.

Pure machine-learning detectors are particularly exposed to the first failure. A classifier trained on recorded movement windows answers one question, namely which of the recorded motions the current window most resembles. It has no notion of how far the body actually tilted, whether the person stayed down afterwards, or what the person was doing before the event. A small lean whose window shape resembles a fall recording can therefore be labelled as a fall even though the person never reached the ground.

The problem addressed in this thesis is therefore: how can a low-cost wearable IMU system recognize falls in real time, avoid raising a confirmed alert on movements that are not falls and explain every decision in terms a caregiver can understand?

1.4 Research gap

The literature on wearable fall detection is large (Chapter 2), but three gaps remain that this work targets. First, many published detectors report a single decision per window, either from thresholds or from a classifier and do not separate the question "does this look like a fall?" from the question "did a fall actually happen?". Second, reported accuracies are often obtained with evaluation protocols in which overlapping windows from the same recording appear in both the training and the test set, which inflates the result and hides how the model behaves on a new recording [5], [6]. Third, explainability is rarely treated as a design requirement of a fall detector: the output is usually a label or a probability, whereas a caregiver needs to know which evidence led to the alert [7].

The gap this thesis fills is a transparent, hybrid pipeline, implemented end-to-end on inexpensive hardware, in which a machine-learning classifier supplies one piece of evidence, physical measurements and post-event behavior supply the others and every alert is accompanied by the evidence that produced it. The thesis also reports the classifier with a session-grouped evaluation protocol so that the quoted accuracy is not inflated by window overlap.

1.5 Aim and objectives

Aim. To design, implement and evaluate a context-aware activity-recognition component for the AegisCare platform that detects falls from a wearable IMU in real time, suppresses false alarms by combining machine-learning and physics-based evidence, and explains each decision to the caregiver.

The aim is broken into the following measurable objectives.

1. **O1.** Build a wearable sensing node that streams calibrated, filtered six-axis motion data over MQTT at about 10 Hz.
2. **O2.** Collect a labelled dataset of six motion classes and train a window-based classifier, comparing at least three algorithms.
3. **O3.** Evaluate the classifier with a protocol that keeps each recording entirely in either the training or the test data.
4. **O4.** Design a stateful confirmation engine that treats a fall as confirmed only when the classifier, the measured body tilt, and the post-event stillness agree.
5. **O5.** Produce a continuous fall-risk score and a plain-language explanation for each decision.
6. **O6.** Integrate the component into the AegisCare dashboard and verify its behavior with scripted test scenarios.

1.6 Research questions or hypotheses

The work is guided by three research questions and two hypotheses.

Table 1.1 Research questions and hypotheses

ID	Statement	Addressed in
RQ1	How accurately can a Random Forest recognize standing, walking and four fall directions from 2-second IMU windows when whole recordings are held out for testing?	Sections 6.1, 7.1
RQ2	Does adding a physical-orientation gate and a stay-down check to the classifier reduce the number of non-fall movements that raise a confirmed alert?	Sections 6.2, 6.4
RQ3	Can the decision be explained to a caregiver using only the quantities the system measures, without a further learned model?	Sections 4.4, 6.2
H1	A session-grouped evaluation gives a lower, more realistic accuracy estimate than a random window split.	Section 6.1

H2	No non-fall recording is confirmed as a fall by the full pipeline, whereas the classifier alone would flag some.	Sections 6.2, 6.4

Source: Author.

1.7 Scope limitations and assumptions

The following boundaries apply to the work and are referred to again when the results are interpreted in Chapter 7.

- **In scope:** one wearable device worn on the trunk, one subject, six pre-defined motion classes, a single broker and a single back-end instance and a web dashboard for a caregiver.
- **Out of scope:** recognition of other activities of daily living (sitting, climbing stairs, lying down), multi-person deployment, automatic escalation to emergency services, and clinical validation with older adults.
- **Assumption A1:** the sensor is worn at a fixed position and orientation, with the Y axis approximately vertical when the person stands upright.
- **Assumption A2:** falls recorded by a healthy subject under controlled conditions are representative enough to develop and compare the pipeline, although they are not a substitute for falls of real older adults.
- **Assumption A3:** Wi-Fi and the cloud broker are available; offline operation on the device is not addressed.

1.8 Individual contribution

The work in this thesis was carried out individually for the Context-Aware Activity Recognition component. Table 1.2 lists the contribution and the artefact in which each item can be inspected. Work on the other AegisCare components belongs to other members of the research group and is described here only where it affects integration.

Table 1.2 Individual contributions and the artefacts that evidence them

Contribution	Artefact
Sensor-node firmware: calibration, EMA filtering, JSON publishing	firmware/FallDetection.ino
Data-collection web tool and labelled dataset (361 recordings)	tools/DataCollector.html; fall_data_*.csv
Feature extraction, model training and comparison (RF, DT, KNN)	FallDetection/train.py

REST service, MQTT subscription, windowing and inference	server.py
Fall Engine: orientation, confirmation state machine, risk score, explanations	fall_engine.py
Posture-decision logic and the eight-section live dashboard	postureEngine.js; Analyzer.jsx; fallApi.js
Session-grouped evaluation and scripted scenario tests (this thesis)	Chapters 3, 5, 6; Appendix A

Source: Author.

1.9 Thesis structure

Chapter 2 reviews the literature on fall detection, human activity recognition and explainable decision support and ends with a synthesis of the gaps. Chapter 3 describes the research methodology, including the dataset, the evaluation protocols and the ethical considerations. Chapter 4 presents the requirements and the design of the component. Chapter 5 describes its implementation, integration, and testing. Chapter 6 reports the results and Chapter 7 discusses them, with their limitations. Chapter 8 considers commercialization and sustainability, and Chapter 9 concludes the thesis and proposes future work. The appendices contain extended results and the disclosure of AI use.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The review covers four bodies of work that meet in this component: the epidemiology and consequences of falls in older adults, body-worn inertial sensing and human activity recognition (HAR), fall-detection algorithms from simple thresholds to machine learning and the evaluation and explainability practices that determine whether a published result can be trusted in deployment.

2.2 Domain and theoretical foundations

2.2.1 Falls in older adults

Falls are a major public-health issue. The WHO identifies them as the second leading cause of unintentional injury death worldwide and notes that people over 60 suffer the largest number of fatal falls [1]. Prospective community studies found that about one-third of people aged 65 and over fall each year and that the risk factors, such as muscle

weakness, gait and balance problems and sedative medication, accumulate [2]. Because a single fall can lead to a long lie on the floor, fear of falling, and loss of independence, systems that detect a fall and summon help quickly are an important complement to prevention.

2.2.2 Inertial sensing

A tri-axial accelerometer measures specific force, the combination of gravity and the acceleration of the body. When the wearer is still, the accelerometer reads the gravity vector, whose length is 1 g (about 9.81 m/s²). The direction of that vector in the sensor frame therefore gives the sensor's orientation, and the tilt angle θ of the trunk from the vertical axis can be computed as in Equation 2.1, where a_γ is the reading along the axis that is vertical when the person stands:

$$\theta = \arccos(a_\gamma / \sqrt{(a_x^2 + a_\gamma^2 + a_z^2)}) \quad (2.1)$$

A tri-axial gyroscope measures angular velocity. Its readings are zero when the body does not rotate, so the spread of the gyroscope signal over a short window is a convenient measure of how much the person is moving. The BMI160 used in this work combines both sensors in one package [8]. Accelerometer-derived tilt is reliable when the body is nearly still but is corrupted by motion, whereas gyroscope readings drift over time; the component therefore uses the accelerometer for orientation and the gyroscope for stillness, which are the two roles for which each is dependable.

2.2.3 Windowed activity recognition

Most sensor-based HAR systems follow the pipeline described by Bulling et al.: the raw signal is segmented into fixed windows, features are computed per window, and a classifier maps the feature vector to an activity label [9]. Typical features are time domain statistics (mean, standard deviation, minimum, maximum, range, root mean square, interquartile range) and correlations between axes. Windows are often overlapped, for example by 50%, to avoid missing events that straddle a boundary. The consequence of overlap for evaluation is discussed in Section 2.3.4.

2.2.4 Random Forest

A Random Forest is an ensemble of decision trees, each grown on a bootstrap sample of the training data with a random subset of features considered at each split; the forest

predicts by majority vote and the fraction of votes can be read as a class probability [4]. It handles correlated features, needs little tuning, is robust to over-fitting compared with a single tree and is cheap enough to run on a small server. These properties, together with the availability of a well-tested implementation [10], make it a reasonable default for small wearable datasets.

2.2.5 Explainability

Explainable artificial intelligence (XAI) distinguishes between models that are interpretable by construction and post-hoc explanations of opaque models [7]. In safety-related decision support the first route is attractive: if the decision rule is built from named, physically meaningful quantities, the explanation is the rule itself and does not need a second model to approximate it. This reasoning underlies the rule based confirmation layer in this thesis.

2.3 Related research

2.3.1 Surveys of fall detection

Several surveys organize the field. Noury et al. describe the principles of fall detection and the typical phases of a fall (pre-fall, impact, and post-fall inactivity) [11]. Mubashir et al. classify approaches into wearable-device, ambience-based, and vision-based methods and discuss their trade-offs [12]. Review the challenges and trends and highlight, among other issues, that most work relies on simulated falls performed by young volunteers and that usability and false alarms limit acceptance [13]. These surveys agree that the post-impact phase carries information that a single acceleration peak does not.

2.3.2 Threshold-based detection

Threshold algorithms compare features such as the acceleration magnitude or the angle of the trunk with fixed limits. Bourke et al. evaluated a threshold-based algorithm using a tri-axial accelerometer on the trunk and reported high sensitivity and specificity on a data set of simulated falls and activities of daily living [14]. Kangas et al. compared several low-complexity algorithms for body-attached accelerometers and found that taking the posture after the impact into account improved detection compared with using the impact alone [15]. Threshold methods are cheap and transparent, but a threshold tuned on one data set may not transfer to another subject

or sensor position, and they have no model of the many movements that resemble a fall.

2.3.3 Machine-learning detection

Özdemir and Barshan compared several machine-learning techniques for fall detection using wearable sensors placed on different parts of the body and reported high detection performance for several classifiers [16]. In general HAR, classical classifiers on hand-crafted features remain competitive on small data sets, while deep networks that learn features directly from the raw signal have become common when large labelled collections are available [17]. Public collections such as the smartphone HAR data set of Anguita et al. have made benchmarking easier, although they describe activities of daily living rather than falls [18].

2.3.4 Evaluation validity

The reliability of a reported accuracy depends on how the data are split. When windows are overlapped and then divided at random into training and test sets, neighboring windows from one recording fall on both sides of the split, so the classifier is partly tested on data it has already seen. Cross-validation strategies that respect the grouping of the data, here the recording session, give a more honest estimate of performance on a new recording [5]. More broadly, data leakage has been identified as a common reason why machine-learning results in applied science fail to reproduce [6]. This thesis therefore reports both the random-split result, which is what the training script produces, and a session-grouped result.

2.4 Existing systems products and techniques

Fall-related products fall into a small number of categories. Table 2.1 compares them qualitatively on the properties that matter for the problem in Section 1.3. The comparison is based on the characteristics of each category reported in the surveys cited above and on general knowledge of how these products work; it is not a benchmark of specific commercial devices.

Table 2.1 Qualitative comparison of fall-detection approaches

Approach	Works without user action	Privacy impact	Typical weakness	Explains decision
Push-button pendant	No	Low	Useless if the person is unconscious	Not applicable

Camera-based monitoring	Yes	High	Intrusive; limited to covered rooms	Limited
Floor or ambient sensors	Yes	Medium	Fixed installation; roombound	Limited
Threshold-based wearable	Yes	Low	False alarms on fall-like movements	Partly (the rule)
ML-only wearable	Yes	Low	Window label without physical check	Rarely
This component (ML + physics + stay-down)	Yes	Low	Single-subject evidence only	Yes, per decision

Source: Author, based on [13], [12].

2.5 Gap synthesis and conceptual framework

The reviewed work shows that the pre-fall, impact, and post-fall phases each carry evidence [11], [15], that classifiers on windowed features can separate falls from walking and standing on a data set of simulated recordings [16], and that reported accuracy depends strongly on the evaluation protocol [5]. What is missing is a compact design that (i) keeps the classifier as one piece of evidence rather than the final judge, (ii) makes the physical and temporal evidence explicit, (iii) states its decision in words, and (iv) is evaluated with a protocol that does not leak recordings between training and test data. Table 2.2 summarises how each gap is closed in this work.

Table 2.2 Identified gaps and how the component addresses them

Gap in the literature	**Design response in this thesis**	**Where**
Single decision per window; no separation of "looks like a fall" from "was a fall"	Three-gate confirmation: classifier ≥ 60%, tilt ≥ 30°, stay-down check	Section 4.4.3
Overlap leakage in random window splits	Session-grouped cross-validation reported next to the random split	Sections 3.3, 6.1
Alerts without explanation	Trigger factors and template-based message built from measured values	Section 4.4.5
Reaction only after the event	Continuous 0–100 risk score that rises before a fall is confirmed	Section 4.4.4
Evidence rarely tied to a deployable system	End-to-end implementation on low-cost hardware with a live dashboard	Chapter 5

Source: Author.

The resulting conceptual framework is the three-gate pipeline shown in Figure 2.1. A window is passed through the classifier gate, the orientation gate and the stay-down gate in turn, and a fall is confirmed only if all three agree. A failure at a gate does not

discard the information: it selects the label (normal activity, leaning, or recovered stumble) that explains why no fall was confirmed.

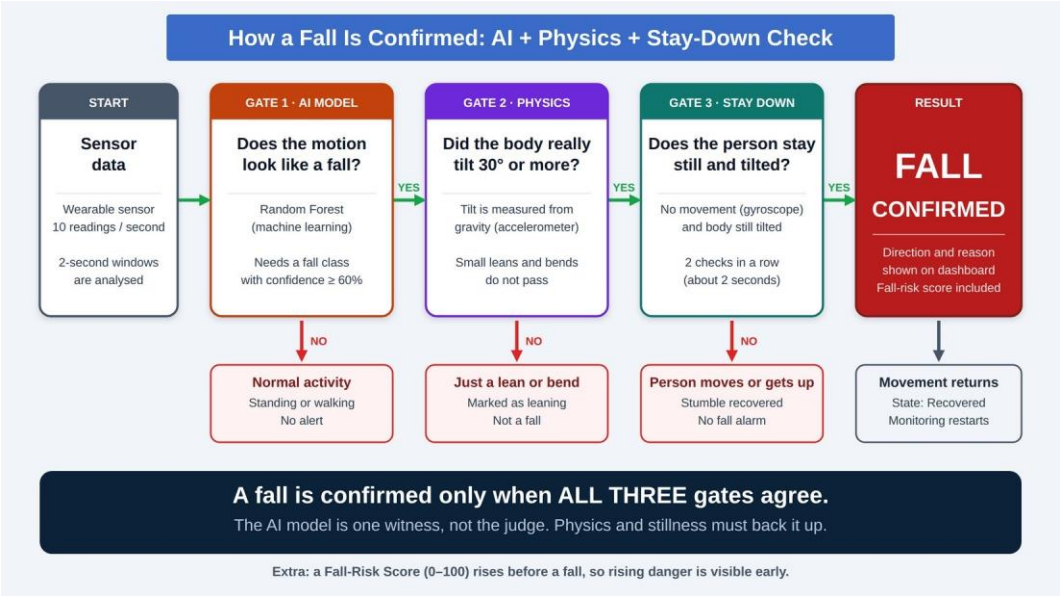


Figure 2.1 Conceptual framework: three gates must agree before a fall is confirmed
Source: Author.

3 RESEARCH METHODOLOGY

3.1 Research design

The research follows a design-science approach combined with a quantitative experimental evaluation. A working artefact, the wearable sensing node with its backend pipeline and dashboard, was designed and built to address the problem in Section 1.3. The artefact was then evaluated at two levels: the machine-learning classifier on its own, and the complete pipeline that adds the confirmation engine. Table 3.1 shows the stages of the study.

The design is a within-data comparison. Every comparison, whether between algorithms, between evaluation protocols, or between the classifier alone and the full pipeline, is made on the same recorded data set, so differences in the reported figures arise from the method and not from the data. This is appropriate for a first-stage study with a single subject, but it limits what can be claimed about other people, which is discussed in Section 7.5.

Table 3.1 Stages of the study and the research question each stage answers

Stage	Activity	Output	RQ
1	Requirements and design of the node, service, engine, and dashboard	Architecture (Chapter 4)	RQ3
2	Data collection of six motion classes with the collector tool	Labelled CSV (Section 3.2)	–
3	Windowing, feature extraction, and training of RF, DT, and KNN	Models and scaler	RQ1
4	Evaluation: random split and session-grouped cross-validation	Table 6.1, Table 6.2, Table 6.3	RQ1
5	Threshold analysis and scripted scenarios for the full pipeline	Table 6.4, Table 6.5, Table 6.6	RQ2
6	Integration check with the dashboard	Section 6.4	RQ2, RQ3

Source: Author.

3.2 Data participants or study materials

3.2.1 Hardware and acquisition

Data were collected with the same sensing node that is used in operation: a BMI160 IMU on a Wemos D1 Mini board, worn on the trunk. The accelerometer range is ±2 g

(16 384 LSB/g) and the gyroscope range is ±2000 °/s (16.4 LSB per °/s). At start-up the firmware averages 200 samples to estimate the sensor offsets, subtracts them, and then applies an exponential moving average with smoothing factor $\alpha = 0.1$ to each axis. The acceleration magnitude is computed as the Euclidean norm of the three accelerometer axes. The data-collection web page receives the MQTT stream, labels each sample with the class chosen by the operator and writes a CSV file with the columns label,person name, session_id, sample_index, timestamp_ms, ax, ay, az, gx, gy, gz, and mag.

3.2.2 Dataset

The data set consists of 14,209 samples in 513 recorded sessions. A session is one continuous recording of one class, for example a single walk or a single forward fall. The six classes are balanced at about 60 sessions each, except standing still, which has 58 sessions and fewer samples because those recordings are shorter. Table 3.2 gives the composition. All sessions come from a single subject on a single day

Table 3.2 Composition of the data set

Class	**Sessions**	**Samples**	**Windows**
Backward fall	61	2,081	121
Forward fall	60	2,076	119
Left fall	61	2,048	116
Normal walk	61	2,144	128
Right fall	60	2,212	141
Standing still	58	1,595	77
Total	**361**	**12,156**	**702**

Source: Author's recordings; window counts computed in this study.

Two properties of the data matter for the interpretation of the results. First, the nominal publish rate is 10 Hz, but the measured interval between consecutive samples has a median of 127 ms, a mean of 123 ms and a 90th percentile of 234 ms, so a 20-sample window spans on average about 2.5 s rather than exactly 2 s. Second, the firmware filtering keeps the acceleration magnitude between 0 and 1.136 g in every class (mean 0.97 g); an impact spike is therefore not visible in the data, and the confirmation engine cannot use an impact threshold with this device (Section 4.4.3).

3.2.3 Software and libraries

The training and evaluation code is written in Python with NumPy, pandas and scikitlearn 1.8.0 [10]. The service uses Flask and the paho-mqtt client and the dashboard is a React 18 application built with Vite. All library versions are fixed in the requirements and package files of the repository.

3.3 Experimental or study procedure

3.3.1 Windowing and feature extraction

Each session is segmented into windows of 20 samples with a step of 10 samples (50% overlap). A window is the unit that is classified. From each window the training script computes seven statistics for each of seven channels (ax, ay, az, gx, gy, gz, mag) and three correlation coefficients between the accelerometer axes, which gives $7 \times 7 + 3 = 52$ features (Table 3.3). Sessions with fewer than 20 samples are zero-padded in the training script; in the evaluation of this thesis they were excluded (22 of the 361 sessions), because a padded window does not represent real motion.

Table 3.3 Feature set computed for every window

Group	**Definition**	**Count**
Per-channel statistics	Mean, standard deviation, minimum, maximum, range, root mean square, interquartile range	7 per channel
Channels	ax, ay, az, gx, gy, gz, mag	7
Cross-axis correlations	Pearson correlation of (ax, ay), (ax, az), (ay, az)	3
Total		**52**

Source: train.py and server.py.

3.3.2 Classifiers

Three classifiers were trained on standardised features (zero mean, unit variance, with the scaler fitted on the training part only): a Random Forest with 200 trees, a Decision Tree grown without a depth limit, and a k-nearest-neighbours classifier with k = 5 and distance weighting. All use a fixed random seed of 42. The Random Forest is the model deployed in the service because it gave the best session-grouped accuracy (Section 6.1).

3.3.3 Evaluation protocols

Two protocols were applied to the same windows. **Protocol P1 (random split)** reproduces the training script: 80% of the windows are used for training and 20% for testing, stratified by class. **Protocol P2 (session-grouped cross-validation)** uses stratified group 5-fold cross-validation (SGKF) with the session as the group, so that all windows of a session are either in the training folds or in the test fold [5]. Results of P2 are reported as the mean and standard deviation over the five folds, and the confusion matrix is built from the out-of-fold predictions of all five folds.

3.3.4 Threshold calibration for the confirmation engine

The orientation thresholds of the Fall Engine were set from the recorded data. For each session the peak tilt angle was computed from the accelerometer (Equation 2.1) and the distribution per class was inspected; the fall-onset threshold of 30° was chosen to lie above the peak tilt of every standing and walking session and below the median peak tilt of the fall sessions (Section 6.2).

3.3.5 Scripted scenarios for the full pipeline

The full pipeline (classifier with the 60% confidence rule, followed by the Fall Engine) was evaluated offline with the recorded sessions. For each held-out session in the P2 folds, the model trained on the other folds produced a prediction for every window in order, and the windows were fed to a fresh Fall Engine. Three scenarios were defined: **stay-down**, in which the windows of a fall session are followed by four windows in which the final posture is held without motion; **stumble**, in which the fall windows are followed by five upright, motionless windows; and **non-fall**, in which the windows of a standing or walking session are followed by four repetitions of the last window. The stay-down and stumble windows are synthetic continuations because the recordings end shortly after the fall, and this is a limitation noted in Section 7.5. Each scenario was run twice: once exactly as the deployed service behaves, in which windows with a classifier confidence below 0.60 are labelled "uncertain" and are not passed to the engine, and once with every window passed to the engine.

3.4 Measures and evaluation metrics

Classifier performance is reported with accuracy, per-class precision, recall, and F1score, and with the weighted F1-score across classes. Because the safety question is

whether a fall is detected at all, the four fall classes are also merged into a single positive class, and **sensitivity** (the fraction of fall windows predicted as any fall class) and **specificity** (the fraction of non-fall windows predicted as a non-fall class) are reported. For the full pipeline the metrics are at the session level: the **confirmation rate** is the fraction of fall sessions in a scenario that reach the state "confirmed fall", and the **false-confirmation rate** is the fraction of non-fall sessions that reach it. The **threshold reach** is the fraction of sessions whose peak tilt is at least 30°.

3.5 Data analysis

All figures were produced with a single script that loads the CSV, builds the windows, and runs P1, P2, the tilt analysis, and the scenarios. Differences between algorithms under P2 are small compared with the fold-to-fold standard deviation, so they are described as indicative and no significance test is claimed: with five folds and one subject, a test would have little power and could suggest more certainty than the data support. The comparison of P1 with P2 for the same model is used to test hypothesis H1, and the comparison of the classifier alone with the full pipeline on non-fall sessions is used to test H2.

3.6 Ethics privacy security and responsible AI

Participants and consent. The recordings contain only inertial sensor values from one adult subject who took part voluntarily; no images, audio, location, or personal identifiers are stored.Falls were simulated in a controlled way, and no older adult was exposed to risk.

Privacy. A wearable IMU is less intrusive than a camera, but motion data can still reveal habits and health status. In a real deployment the data would be personal data and would fall under the Personal Data Protection Act, No. 9 of 2022 of Sri Lanka [19], which requires a lawful basis, purpose limitation, and appropriate security. The current prototype writes only one record per monitoring session to the database (the wearer's name as typed by the caregiver and the start time) and does not store the raw sensor stream, which exists only in memory; even so, the name is personal data and the table would need access control and a retention rule.

Security. The service connects to the cloud broker over TLS on port 8883. The security review for this thesis nevertheless identified weaknesses that are acceptable for a

research prototype but not for deployment: broker credentials and the dashboard demonstration login are present in the source code, the firmware and the service are configured for different brokers, and the REST endpoints have no authentication. These are listed as limitations in Section 7.5 and as required work in Section 9.4.

Responsible AI. A false negative (a missed fall) is more harmful than a false positive, but a system that raises too many false alarms is switched off. The design therefore reports uncertainty explicitly: a window whose classifier confidence is below 60% is labelled "uncertain" and never raises an alert on its own. The system supports a caregiver, it does not replace one, and the thesis makes no claim of clinical validity.
Use of generative AI tools during the preparation of this thesis is disclosed in Appendix B.

3.7 Reproducibility and research artefacts

The artefacts needed to repeat the study are the recorded CSV file, the training script (train.py) with its fixed seed, the trained model, scaler and label-encoder files, the evaluation script used for this thesis, and the source code of the firmware, service, and dashboard. The evaluation uses scikit-learn 1.8.0 with random_state = 42 for the models and for the fold assignment. The main figures in Chapter 6 can be regenerated by running the evaluation script on the CSV file.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements were derived from the problem statement, from the constraints of the AegisCare platform, and from the properties that a caregiver needs in order to trust an alert. They are grouped into functional requirements (FR) and non-functional requirements (NFR) in Table 4.1.

Table 4.1 Functional and non-functional requirements

ID	Requirement	Verified by
FR1	The node shall stream calibrated six-axis motion data at about 10 Hz to a cloud broker.	Section 5.4 (T1)
FR2	The service shall classify each 20-sample window into one of six motion classes with a confidence value.	Section 6.1
FR3	A fall shall be reported as confirmed only when the classifier, the body tilt, and the stay-down check all agree.	Section 6.2

FR4	A window with classifier confidence below 60% shall be labelled "uncertain".	Section 5.4 (T4)
FR5	The system shall compute a continuous fall-risk score from 0 to 100.	Section 5.4 (T6)
FR6	Every decision shall be accompanied by a plain-language explanation and its trigger factors.	Section 5.4 (T6)
FR7	The dashboard shall show posture, risk, orientation, movement, and connection state live.	Section 6.4
NFR1	End-to-end update interval of the dashboard shall not exceed about 1 s.	Section 6.4
NFR2	The component shall run on a low-cost node and a single small server.	Section 5.5
NFR3	The decision logic shall be inspectable (named thresholds, no hidden state beyond the documented state machine).	Section 4.4
NFR4	Communication from the node to the broker and from the broker to the service shall be encrypted.	Section 7.5

Source: Author.

4.2 Component architecture within the complete solution

The component has five layers (Figure 1.1). The **sensing layer** is the BMI160 and the ESP8266 firmware. The **transport layer** is MQTT over TLS to a HiveMQ Cloud broker, using the topic elder/device01/imu. The intelligence **layer** is a Flask service on port 5050 that subscribes to the topic, keeps the latest sample, and exposes four REST endpoints. The **decision layer** is the Fall Engine inside the service. The **presentation layer** is the React dashboard, which polls the service every 100 ms and also runs its own posture-decision logic. Table 4.2 lists the interface that the component exposes to the rest of AegisCare.

Table 4.2 REST interface of the activity-recognition service (port 5050)

Endpoint	Method	Purpose
/health	GET	Reports that the service and model are loaded, the window size, and the broker connection state
/latest	GET	Returns the most recent sensor sample received over MQTT
/predict	POST	Accepts a window of 20 × 7 values, returns the label, confidence, class probabilities, the Fall Engine result and the message
/reset	POST	Resets the Fall Engine state machine

Source: server.py.

Within the platform the component behaves as an independent service. The other AegisCare services never read its internal state; they consume only its REST output, which keeps the coupling low and allows each component to be developed and tested separately.

4.3 Design alternatives and decisions

Table 4.3 records the main design decisions, the alternatives that were considered, and the reason for the choice. The decisions follow from three priorities: a low-cost device, a transparent decision rule, and a result that can be explained and tested with small data.

Table 4.3 Design alternatives and decisions

Decision	**Alternatives considered**	**Choice and reason**
Where to run inference	On the microcontroller; on a server	Server. The ESP8266 has little memory, and a server allows the model and thresholds to be changed without re-flashing the device.
Classifier	Decision Tree, KNN, deep network	Random Forest. Best grouped accuracy of the three classical models, probability output for the confidence rule, small training set makes a deep network unattractive.
Final decision	Classifier alone; thresholds alone; combination	Combination. The classifier alone mislabels fall-like movements; thresholds alone ignore movement pattern; the combination lets each correct the other.
Stay-down evidence	Impact spike; post-fall stillness	Stillness. The firmware filtering removes impact peaks (maximum magnitude 1.14 g), so an impact threshold cannot be used.
Transport	HTTP polling of the device; MQTT publish/subscribe	MQTT. Lightweight, designed for constrained devices, and decouples the node from the service [3].
Dashboard update	WebSocket push; HTTP polling	Polling every 100 ms. Simple, stateless, and sufficient for a 10 Hz stream.
Explanation	Post-hoc XAI method; learned text model; templates	Templates filled with measured values. Interpretable by construction, deterministic, and testable [7].

Source: Author.

4.4 Detailed design

4.4.1 Sensor node and message format

The firmware performs five steps in a loop: read the BMI160 over the I²C bus (address 0x69, SDA on D2 and SCL on D1), convert raw counts to g and °/s, subtract the offsets measured during the 200-sample start-up calibration, smooth each axis with the exponential moving average $y_t = \alpha\, x_t + (1 - \alpha)\, y_{t-1}$ with $\alpha = 0.1$, and publish a JSON

object containing the six axes and the magnitude, then wait 100 ms. The smoothing suppresses sensor noise, which suits the posture decision, but it also removes sharp acceleration peaks, which is why an impact check is not available (Section 4.3).

4.4.2 Feature extraction and classifier

The service stores only the latest sample received over MQTT. The dashboard polls /latest every 100 ms, keeps the most recent 20 new samples, and posts them to /predict every 10 new samples, which gives the same 50% overlap as in training. For each posted window the service computes the same 52 features as the training script (Section 3.3.1), applies the saved scaler, and obtains class probabilities from the Random Forest. The label is the class with the highest probability, and that probability is the confidence. If the confidence is below 0.60 the label is replaced by "uncertain". Using the same function for training and inference avoids training–serving skew.

4.4.3 Fall Engine: three-gate confirmation

The Fall Engine receives the label, the confidence, the class probabilities, and the raw window. It first computes physical signals from the window: the tilt angle of every sample (Equation 2.1), the peak and final tilt, the dominant direction (forward, backward, left, or right, from pitch and roll), the gyroscope energy (the sum of the standard deviations of the three gyroscope axes) and a motion energy that adds the spread of the accelerometer axes. It then applies the state machine in Figure 4.1 with the thresholds in Table 4.4.

Table 4.4 Thresholds of the Fall Engine

Parameter	Value	Meaning
CONFIDENCE_THRESHOLD	0.60	Minimum classifier confidence; below it the window is "uncertain"
LEAN_ONSET_DEG	15°	Peak tilt from which the engine reports "leaning"
FALL_ONSET_DEG	30°	Peak tilt needed, together with a fall label, to start a possible fall
FALLEN_ORIENTATION_DEG	25°	Tilt that must still be present at the end of the window while pending
STILLNESS_GYRO_ENERGY	8.0	Gyroscope energy below which the body is considered still
REQUIRED_STILL_CONFIRMATIONS	2	Consecutive still and tilted windows needed to confirm
MAX_PENDING_CYCLES	4	Windows allowed in the pending state before a stumble is declared recovered

RECOVERY_MOTION_ENERGY	12.0	Motion energy that ends a confirmed fall (the person moves again)
IMPACT_MAG_G	None	Impact check, disabled because of the firmware smoothing

Source: fall_engine.py and server.py.

The engine has three states. In monitoring it looks for a fall label together with a peak tilt of at least 30°; if both hold the state becomes **PENDING** and the event is "possible fall". If only the tilt reaches 15° the event is "leaning". In **PENDING** each new window is checked for stillness (gyroscope energy below 8.0) and a final tilt of at least 25°; two consecutive such windows move the state to **CONFIRMED** and the event is "confirmed fall". If four windows pass without confirmation, the engine returns to MONITORING with the event "near-fall recovered". In **CONFIRMED** the engine keeps reporting the fall until the motion energy exceeds 12.0, which is taken as the person getting up or being helped, and the event becomes "recovered from fall".

The engine therefore implements the three gates of Figure 2.1 as follows: gate 1 is the classifier label and confidence, gate 2 is the peak tilt, and gate 3 is the stay-down check in the pending state. An important detail is that, in the deployed service, a window labelled "uncertain" is not passed to the engine, so it neither advances nor resets the state machine. The consequences are analyzed in Sections 6.2 and 7.5.

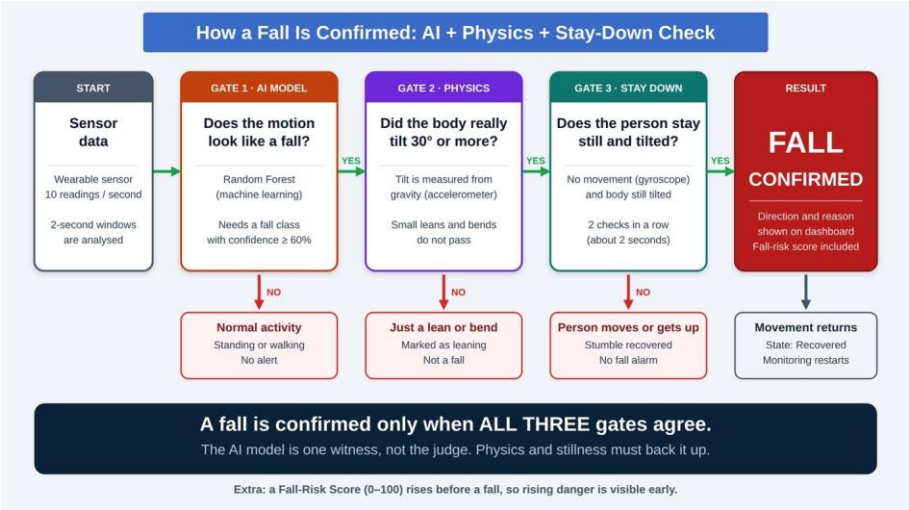


Figure 4.1 Fall Engine decision flow: the three gates and the outcome of each failure *Source: Author; same diagram as Figure 2.1.*

4.4.4 Fall-risk score

The risk score is a transparent heuristic in the range 0 to 100 and is computed for every window that is passed to the engine. It is the sum of three components, as in Equation

4.1: an *instability* component that grows with the motion energy and saturates at a motion energy of 3.0 (maximum 45 points), a *lean* component that grows with the mean tilt and saturates at 45° (maximum 30 points), and a *classifier* component equal to 25 times the total probability that the classifier assigns to the four fall classes.

$$R = 45 \cdot \min(E/3,\ 1) + 30 \cdot \min(\theta</45°,\ 1) + 25 \cdot \min(\Sigma\, p_fall,\ 1) \qquad (4.1)$$

Scores of 40 or more are labelled "elevated" and scores of 70 or more "high". The weights are design choices and have not been validated against clinical fall-risk measures; the score is presented as an early-warning indicator for movement that is becoming unstable, not as a clinical risk assessment.

4.4.5 Explanations

Each result carries a list of **trigger factors**, for example "Body tilt reached 38° (backward)" or "No recovery motion for about 2 consecutive checks after impact", and a sentence assembled from fixed templates, which states the direction, the activity before the event, and the evidence. The engine remembers the last non-fall activity, so the message can say that a person "fell backward while walking normally". The service contains a hook for a language-model rewrite of the message, but it is not used; all explanations in this thesis come from the templates.

4.4.6 Posture-decision logic in the dashboard

The dashboard contains a second, independent decision layer in the module postureEngine.js. It evaluates the tilt angle, the tilt speed, and the angular velocity over time and assigns one of six levels (Table 4.5). The two layers use compatible thresholds: both consider a person to have fallen only after a tilt of about 30° or more followed by stillness, and the 45° mark on the dashboard gauge is the trigger of Level 4 (a potential fall) rather than the confirmation threshold. Level 5 requires the stay-down check of 3 s, and a fall label from the classifier counts as evidence only above 60% confidence and never raises an alert on its own.

Table 4.5 Posture levels of the dashboard decision logic

Level	Name	Condition (summary)
0	Upright	Tilt below 10°
1	Slight tilt	Tilt of 10°–25° with steady movement
2	Losing balance	Tilt of 25° or more with unstable movement

3	Early fall warning	Fast tilting (≥ 12 °/s) with high angular velocity (≥ 11 °/s), at any angle of at least 10°
4	Potential fall	Tilt of 45° or more with fast downward movement and a sudden posture change
5	Fall confirmed	After Level 3 or 4, the body stays tilted by at least 30° and still (below 3 °/s) for 3 s

Source: postureEngine.js (thresholds in the source).

4.4.7 Dashboard

The dashboard presents the output in eight sections (Table 4.6). The first, the posture decision, is the primary result and is shown in Figure 4.2; the others provide the evidence behind it.

Table 4.6 Sections of the live dashboard

No.	Section	Content
1	Posture Decision	Gauge of tilt angle, level label, one-sentence explanation, tilt, angular velocity, tilt speed, movement
2	Body Orientation	Three-dimensional avatar with pitch, roll, and direction
3	Movement Magnitude	Live acceleration magnitude in g
4	Live Prediction	Classifier label and the probability of each of the six classes
5	Context Awareness	Previous activity, direction, state, explanation with trigger factors, and fall-risk score
6	Sensor Data	Live values of the six axes and the magnitude
7	Prediction History	Recent classifier outputs in the current session
8	Posture Event Log	Time-stamped log of posture-level events

Source: Analyzer.jsx.

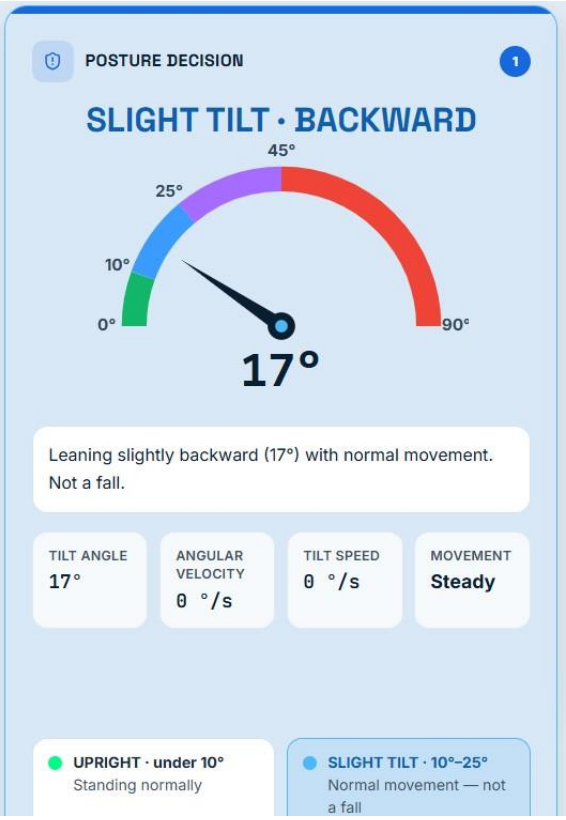


Figure 4.2 Posture-decision section of the dashboard showing a slight backward tilt of 17° ("not a fall")

Source: Screenshot of the implemented dashboard.

4.5 Standards and professional practice

The design follows established standards where they exist. Messages use MQTT 3.1.1 [3] over TLS. The service exposes a resource-oriented REST interface in the style described by Fielding [20]. The BMI160 is operated within the ranges given by its data sheet [8]. Quality attributes in Table 4.1 are named after the product-quality model of ISO/IEC 25010 [21]: functional suitability (FR1–FR6), performance efficiency (NFR1, NFR2), maintainability (NFR3), and security (NFR4). Personal-data handling is considered against the Sri Lankan Personal Data Protection Act [19]. Source code is version-controlled, the thresholds are collected in one class, and each module has a single responsibility, which supports the maintainability requirement.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

Table 5.1 lists the technologies used in each layer. The choice favours widely available, low-cost tools so that the component can be reproduced in a student laboratory.

Table 5.1 Technology stack of the component

Layer	Technology	Role
Sensor	Bosch BMI160 6-axis IMU (±2 g, ±2000 °/s)	Acceleration and angular velocity
Microcontroller	Wemos D1 Mini (ESP8266), Arduino framework, I²C	Calibration, filtering, Wi-Fi, MQTT publish
Broker	HiveMQ Cloud, MQTT over TLS (port 8883)	Message transport
Service	Python, Flask, flask-cors, paho-mqtt	REST API and MQTT subscription
Machine learning	scikit-learn 1.8.0, NumPy, pandas	Features, Random Forest, scaler, label encoder
Decision logic	Python (fall_engine.py); JavaScript (postureEngine.js)	Confirmation, risk, explanations; posture levels
Dashboard	React 18, Vite, Tailwind CSS, Framer Motion, Three.js	Live visualisation and 3-D avatar
Storage	Supabase (table monitoring_sessions)	Log of monitoring sessions only

Source: Repository configuration files.

5.2 Component implementation

5.2.1 Firmware

The firmware (about 140 lines of Arduino C++) initializes the I²C bus, configures the BMI160 ranges, and runs the 200-sample calibration while the device is held still. In the main loop it reads the six axes, converts them to physical units, subtracts the calibration offsets, applies the exponential moving average with $\alpha = 0.1$, computes the magnitude, builds a JSON string, publishes it to elder/device01/imu, and delays 100 ms. If the broker connection is lost, the loop retries every two seconds until it is restored.

5.2.2 Service

The service (about 415 lines of Python) loads the model, the scaler, and the label encoder at start-up, connects to the broker over TLS, subscribes to the topic, and stores the most recent message. /predict validates the request (a JSON body with a window array of 20 samples with seven numeric fields each), builds the feature vector, applies

the scaler, obtains the probabilities, applies the 60% confidence rule, calls the Fall Engine, assembles the response, and returns it as JSON. The response contains the label, the confidence, the probability of every class, the engine result (event, state, direction, risk score and level, trend, physical signals, trigger factors, and previous activity), the human-readable message, and a fall - alert flag that is true while an event is a possible, confirming, or confirmed fall. A single Fall Engine instance holds the state machine, and /reset clears it.

5.2.3 Fall Engine

The engine (about 490 lines) is a class with the thresholds in a separate class, so that they can be changed in one place. The state machine described in Section 4.4.3 is implemented in the update method. Because the engine is stateful, its behavior depends on the order of the windows, which is why the tests in Section 5.4 replay whole sessions rather than isolated windows.

5.2.4 Dashboard

The dashboard page (about 1 050 lines of React) polls /latest, assembles windows, calls /predict, feeds each new sample to the posture engine, and renders the eight sections. Color and motion are used to encode severity: the gauge changes color with the level, the posture-decision card turns red when a fall is confirmed, and the 3-D avatar leans in the direction of the measured tilt. Sampling and prediction rates are defined as constants (POLL_INTERVAL_MS = 100, PRED_EVERY = 10, WINDOW_SIZE = 20) in one file, and the window size must match the service and the training script.

5.3 Integration with the other components

The component is integrated with the rest of AegisCare through the web application. The home page lists the four modules; the activity-recognition card leads, after the platform login, to a short form that asks who is wearing the device, and then to the dashboard. The base URL of the service is read from an environment variable (VITE_API_BASE_URL), with http://127.0.0.1:5050 as the default, so the dashboard can be pointed to another host without a code change. When the caregiver enters the wearer's name and starts monitoring, the dashboard writes one record (the name and the start time) to the table monitoring_sessions, which is the only use of the database. Each of the other back-end services (cardiovascular, environmental risk, alerts) runs

as a separate Flask process; the activity-recognition service does not call them, and the smart-alert component is intended to consume its output through the REST interface.

5.4 Verification and software testing

Testing was carried out at four levels. **API tests** exercised the real server.py through the Flask test client, with the MQTT client replaced by a stub because no broker is reachable from the test environment. **Engine scenarios** replayed recorded sessions through the Fall Engine (Section 3.3.5). **Dashboard-logic scenarios** replayed the same sessions through the JavaScript posture engine. **Hardware tests** concern the physical node and must be run on the device. Table 5.2 lists the tests; the results are given in Chapter 6 and in Appendix A.

Table 5.2 Test cases

ID	Module	Test	Expected result	Status
	T1	Node publishes calibrated samples at about 10 Hz (hardware)	JSON messages on the topic; offsets removed	Evidenced by the recorded data (Section 3.2.2); live log [to attach]
	T2	GET /health, GET /latest, POST /reset	HTTP 200 with the documented body	Executed
	T3	POST /predict with real standing and walking windows	Correct label, high confidence, event "normal"	Executed
	T4	POST /predict with noise, a short window, no window, and no body	"uncertain" for noise; HTTP 400 for invalid input	Executed
	T5	Replay of a real forward-fall session through /predict	"possible fall", then "confirming"	Executed
	T6	Risk score and explanation in every response	Score in 0–100; message and trigger factors present	Executed
	T7	Engine scenarios: stay-down, stumble, non-fall	Confirmed, not confirmed, not confirmed	Executed (Section 6.2)
	T8	Dashboard posture engine: fall then stay down; fall then recover; non-fall; slow bend	Level 5; never 5; below Level 3; below Level 3	Executed (Section 6.4)

Source: Author.

Tests T2 to T6 passed. The health endpoint returned the six class names and the window size of 20. Standing and walking windows were labelled "standing_still" (confidence 0.995) and "normal_walk" (0.965) with the event "normal". A window of

random noise returned the label "uncertain" with a confidence of 0.3. A window of the wrong length, a request without a window, and a request with an empty JSON body returned HTTP 400, and the replay of a real forward-fall session produced the events "possible fall" (tilt 40° forward) and then "confirming", with a risk score of 84.3 rising to 95.3.

One defect was found. A POST /predict request that has no JSON content type at all returns HTTP 500 instead of 400, because the exception raised by the JSON parser is caught by the general handler. This does not affect the dashboard, which always sends JSON, but it is recorded in Section 7.5 and should be corrected.

The testing is limited in three respects. The tests T3 to T5 use windows that were part of the training data of the deployed model, so they check the plumbing and not the accuracy; accuracy is measured separately with held-out sessions in Section 6.1. There is no automated regression suite in the repository, so the scripts used here would have to be added to it. And T1 requires the physical node: the recorded data show that it works, but a log of live operation is not attached to this thesis.

5.5 Deployment and operation

The component is deployed locally for demonstration. A start script installs the Python requirements and runs server.py, which listens on port 5050, and a second script runs the Vite development server for the dashboard. The node is powered over USB and connects to a Wi-Fi network whose credentials are set in the firmware. The broker is a hosted service, so no broker needs to be operated by the user.

Two configuration issues must be resolved before any real deployment. First, the firmware is configured with one cloud broker (an EMQX endpoint on port 1883, without TLS) while the service subscribes to a HiveMQ Cloud broker on port 8883 with TLS; the two must be set to the same broker for the stream to reach the service. Second, the credentials are present in the source files; they should be moved to environment variables, which the service already supports for the broker settings, and rotated.

6 RESULTS AND VALIDATION

6.1 Research results

6.1.1 Classifier accuracy under two evaluation protocols

Table 6.1 gives the result of the random 80/20 split (protocol P1) for the three algorithms.

Table 6.1 Results of the random 80/20 split (protocol P1)

Model	Accuracy (%)	Weighted F1 (%)
Random Forest		
Decision Tree		
KNN		1

Source: Author's evaluation of the recorded data set.

Table 6.2 gives the result of the session-grouped cross-validation (protocol P2). Accuracy falls for every model, from % to between % and %, and the fold standard deviation is about 1.5 to 1.8 percentage points. The Random Forest has the highest mean accuracy and F1-score, but its advantage of about 0.6 percentage points over the other two models is smaller than one standard deviation, so the ranking should be read as indicative.

Table 6.2 Results of session-grouped 5-fold cross-validation (protocol P2), mean ± standard deviation

Model	Accuracy (%)	Weighted F1 (%)
Random Forest		
Decision Tree		
KNN		

Source: Author's evaluation of the recorded data set.



Figure 6.1 Accuracy of the three models under the random split and under session-grouped cross-validation

Source: Author's evaluation.

The difference between Figure 6.1's two bars for the Random Forest, 100% against 98.15%, is the measured cost of window overlap in this data set. Hypothesis H1 is therefore supported: the session-grouped estimate is lower than the random-split estimate. The deployed model was trained on all windows, so its accuracy on a new recording is best estimated by the grouped figure, not by the training script's own output.

6.1.2 Per-class performance and confusions

Table 6.3 gives the per-class results of the Random Forest built from the out-of-fold predictions, and Figure 6.2 shows the confusion matrix. The four fall classes are recognized with F1-scores between % and %. The weakest class is standing still (F1 = 93.4%), which is mostly confused with walking (6 windows) and which also draws in four backward-fall windows. Left falls are recognized without error; forward falls have no missed windows but one backward-fall window is labelled forward.

Table 6.3 Per-class results of the Random Forest (out-of-fold predictions)

Class	Precision (%)	Recall (%)	F1 (%)	Windows
Standing still		92.2	93.4	77
Normal walk		99.2	96.9	128
Left fall		100.0	100.0	116

Right fall	100.0	99.3	99.6	141
Forward fall	99.2	100.0	99.6	119
Backward fall	99.1	95.9	97.5	121

Source: Author's evaluation.

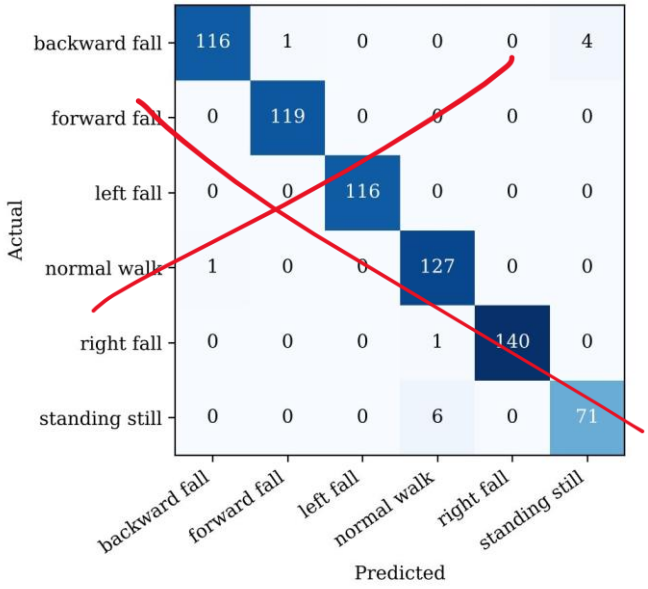


Figure 6.2 Confusion matrix of the Random Forest under session-grouped cross-validation (counts of windows)

Source: Author's evaluation.

When the four fall classes are merged, the sensitivity is 98.99% (492 of 497 fall windows are predicted as some fall class) and the specificity is 99.51% (204 of 205 non-fall windows are predicted as non-fall). The five missed fall windows are four backward falls labelled as standing and one right fall labelled as walking; the single false alarm is a walking window labelled as a backward fall. These values describe the window-level classifier and must not be confused with the session-level confirmation results in Section 6.2.

6.1.3 Feature importance

Figure 6.3 shows the ten most important features of the Random Forest trained on all windows. The leading features are the minimum, mean, and maximum of the ax and az axes and the correlation between ax and ay. These are posture features, since ax and az change when the trunk tilts forward, backward, or sideways, which is consistent with the finding that the model separates the four fall directions well. The gyroscope appears only once (the mean of gz), so the classifier relies mostly on orientation, which

is also what the tilt gate measures; the evidence from the classifier and from the tilt gate is therefore not fully independent, a point taken up in Section 7.1.

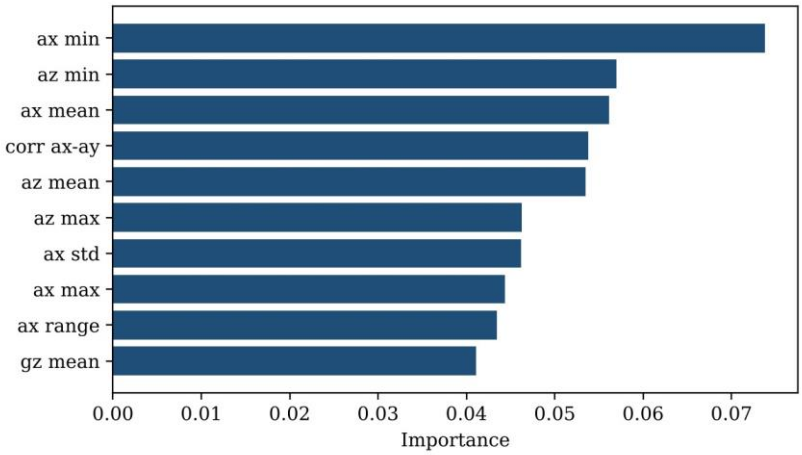


Figure 6.3 Ten most important features of the Random Forest (impurity-based importance)
Source: Author's analysis.

6.2 Product and technical results

6.2.1 Tilt angle by class and the choice of the 30° gate

Table 6.4 gives the peak tilt per session by class, computed from the accelerometer with Equation 2.1, and Figure 6.4 shows the distributions with the 30° gate. Standing sessions never exceed 8.6° and walking sessions never exceed 12.1°, whereas the median peak tilt of the four fall classes lies between 35.6° and 42.5°. A gate at 30° lies well above every non-fall session and below the median of every fall class.

Table 6.4 Peak tilt per session (degrees)

Class	Median	Minimum	Maximum
Standing still	3.7	3.1	8.6
Normal walk	5.6	2.2	12.1
Left fall	35.6	18.9	50.2
Right fall	42.5	24.1	61.8
Forward fall	37.6	27.0	51.6
Backward fall	38.7	22.5	66.7

Source: Author's analysis of 361 sessions.

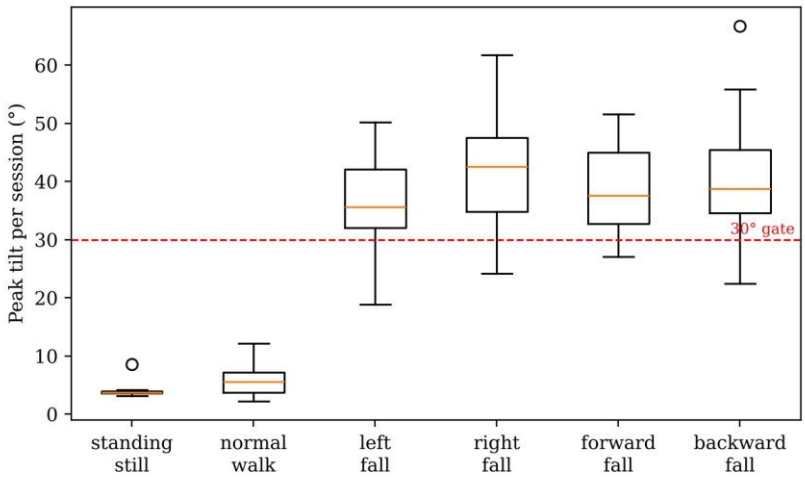


Figure 6.4 Distribution of the peak tilt per session by class with the 30° gate
Source: Author's analysis.

The gate has a cost. Of the 242 fall sessions, 88.4% (214) reach 30°, so about one in nine recorded falls never passes the orientation gate. The lowest fall peaks are 18.9° (left), 22.5° (backward), and 24.1° (right). None of the 119 non-fall sessions reaches 30°. Lowering the gate would recover more falls but would reduce the margin to the maximum of walking (12.1°); the 30° setting is a trade-off between missed falls and false alarms and has been set on one subject only.

6.2.2 Full-pipeline scenarios

Table 6.5 gives the session-level results of the scripted scenarios of Section 3.3.5. The numbers of sessions are 240 fall sessions and 99 non-fall sessions, after the 22 sessions shorter than 20 samples were excluded.

Table 6.5 Session-level results of the full pipeline (classifier, 60% rule, Fall Engine)

Scenario	**Outcome**	**Every window passed to the engine**	**Deployed behaviour (uncertain windows skipped)**
Stay-down (240 fall sessions)	Confirmed fall	196 (81.7%)	49 (20.4%)
Stumble (240 fall sessions)	Recovered, not confirmed	172 (71.7%)	172 (71.7%)
Stumble (240 fall sessions)	Confirmed before recovery	24 (10.0%)	24 (10.0%)
Non-fall (99 sessions)	Confirmed fall	0 (0.0%)	0 (0.0%)
Non-fall (99 sessions)	Any "leaning" or "possible fall" event	0 (0.0%)	0 (0.0%)

Source: Author's scripted scenarios.

Three results stand out. First, no non-fall session is confirmed as a fall, and none even produces a "leaning" or "possible fall" event. Second, in a stay-down scenario the engine confirms 81.7% of the fall sessions when every window reaches it, which is the fraction of sessions in which the classifier and the 30° gate both fire; the remaining 18.3% never enter the pending state. Third, the deployed behavior confirms only 20.4%. The reason is that the engine is bypassed for windows with a classifier confidence below 60%, and 820 of the 960 stay-down windows (85.4%) were labelled "uncertain". A motionless window at a large tilt is unlike any training window, because the training falls contain motion, so the classifier is not confident about it, and the confirmation counter, which needs two consecutive still windows, does not advance. This is an important finding about the interaction of the confidence rule and the stay-down check. It is a result on synthetic stay-down windows, so it should be confirmed with recordings of real people lying still, but it shows that the confirmation logic must not depend on the classifier for windows in which the person is no longer moving. The change that follows is simple (pass every window to the engine, or add a "lying still" class) and is listed as future work in Section 9.4.

The stumble scenario shows that the stay-down gate does what it is meant to do for most sessions: in 71.7% the person is classified as having recovered. In 10.0% the fall is confirmed before the upright windows arrive. This is a limitation of the test and not of the engine, because those recordings already contain the person lying still before the end of the recording, so the fall had in effect been confirmed already.

6.3 User or stakeholder validation

No user study with older adults, caregivers, or clinicians was carried out for this thesis, so the validation of the component rests on the technical tests in this chapter. This is a gap, because the acceptability of the alerts, the readability of the explanations, and the burden of false alarms can only be judged by the people who would use the system. A proposed validation protocol is given in Section 9.4: a think-aloud session with caregivers who are shown recorded scenarios and asked to say what they believe happened and how confident they are, followed by a comparison of the answers with the system's explanation.

6.4 Integration results

The API tests in Section 5.4 show that the service returns well-formed responses to the dashboard and that its three kinds of output (label, engine result, message) are consistent. The update interval of the classifier-based decision is set by the rule of one prediction per ten new samples. With the measured mean sample interval of 123 ms, that is about 1.2 s, which is slightly above the target of about 1 s in NFR1. The posture engine in the dashboard, in contrast, is updated with every new sample, about every 123 ms, and so reacts faster than the classifier layer. A measurement of the end-to-end latency from the sensor to the screen was not made and is a useful addition.

Table 6.6 gives the results of replaying the recorded sessions through the dashboard's posture engine. In the fall scenarios the classifier evidence was supplied as the true fall label with a confidence of 0.9, so that the result shows what the posture logic does when the classifier is right; it is an upper bound for the classifier-dependent path.

Table 6.6 Results of the dashboard posture engine on the recorded sessions

Scenario	Sessions	Reached Level 5 (fall confirmed)	Other outcome
Fall, then stays down 4 s	240	210 (87.5%)	1 more reached Level 4
Fall, then upright and still	240	0 (0.0%)	Event cancelled on recovery
Standing or walking, then still	99	0 (0.0%)	0 reached Level 3 or higher
Slow bend to 50° and hold (script)	1	0 (0.0%)	Highest level reached: 1

Source: Author's scripted scenarios.

The posture engine confirms 87.5% of the falls, compared with 81.7% for the server engine when it sees every window, and never confirms a fall for a recovered stumble, for walking and standing, or for a deliberate slow bend. Its higher confirmation rate comes from the fact that it can react to fast tilting and high angular velocity without a classifier label, and that it works on every sample instead of on windows. The two layers are therefore complementary: the dashboard layer is more sensitive in these tests, and the server layer provides the explanation and the risk score.

6.5 Summary of findings

Table 6.7 Summary of findings by research question

Question	Finding	Evidence

RQ1	Random Forest reaches 98.15 ± 1.71% window accuracy with whole sessions held out, with fall sensitivity 99.0% and specificity 99.5%.	Table 6.2, Table 6.3
RQ2	No non-fall session is confirmed or even flagged. About 18% of fall sessions never pass the 30° gate or the classifier, and the deployed confidence rule weakens the stay-down check.	Table 6.4, Table 6.5
RQ3	Each decision carries measured quantities and a template message, with no further learned model. Readability for caregivers is not yet tested.	Section 4.4.5; Section 6.3
H1	Supported: 100% (random split) against 98.15% (grouped).	Table 6.1, Table 6.2
H2	Partly supported: the full pipeline confirms no non-fall session, but the classifier alone also made only one false fall window among 205, so the benefit of the gates against real confounders was not demonstrated.	Table 6.3, Table 6.5

Source: Author.

7 DISCUSSION

7.1 Interpretation of findings

The results support the central design idea of the component: a machine-learning classifier is a good source of evidence about *what kind* of motion occurred, but it is not enough on its own to decide whether a fall happened. The classifier separates the six recorded classes with 98.15% accuracy on unseen recordings, and the most important features are the posture features of the accelerometer. Because those features also determine the tilt angle, the classifier and the orientation gate are not independent witnesses: when the classifier says "forward fall" the tilt is usually large, and when the tilt is large the classifier usually says "fall". The practical value of the second gate is therefore less to add information than to give a physical, explainable reason and a fixed boundary below which an alert is never confirmed.

The stay-down gate adds information of a different kind, namely what happens after the movement. The scenario results show that it distinguishes a person who remains down from one who recovers in most sessions, and the dashboard-level engine confirms 87.5% of falls without confirming any recovered stumble. The unexpected finding is that in the deployed server pipeline the stay-down check is weakened by the 60% confidence rule, because a motionless posture is an unfamiliar input for a classifier trained on moving falls. This finding illustrates a general point: rules that are safe in isolation, such as "ignore low-confidence predictions", can interact badly with a stateful rule that needs consecutive observations.

The comparison of the evaluation protocols shows how easily an accuracy of 100% can be obtained and how little it means. With 50% window overlap and a single subject, a random split lets the model be tested on near-copies of its training data. The grouped estimate of about 98% is a more honest figure, though it is still optimistic for a new person, because all sessions come from one subject, and for a different sensor placement.

7.2 Comparison with literature baselines and existing solutions

Threshold-based detectors with a trunk-worn accelerometer have been reported to reach high sensitivity and specificity on simulated falls [14], and machine-learning detectors on wearable data have reported high figures as well [16]. The accuracy of

….% obtained here is in the same range, but a numerical comparison would be misleading for three reasons: the data sets differ in subjects, sensor positions, and fall types; the other studies have different class definitions, and in the present data the nonfall classes are only standing and walking, which are easy to separate from falls; and the evaluation protocols differ. What can be compared with the literature is the design. Kangas et al. found that post-fall posture information improves detection over impact alone [15], and the stay-down gate and the tilt gate in this component embody the same principle. The surveys' criticism that explanations and usability receive too little attention [13], [12] is addressed in the design here, although usability has not yet been tested.

Against the categories in Table 2.1, the component keeps the advantages of a wearable (no user action, low privacy impact, no fixed installation) and adds an explicit explanation and a confirmation step. It does not yet offer the clinical evidence, the battery life, the waterproofing, or the cellular connectivity of commercial devices, and it should be regarded as a research prototype.

7.3 Research contribution

- A three-gate confirmation design in which a classifier label, a measured tilt angle, and a stay-down check must agree before a fall is confirmed, with all thresholds named and set from data.
- A protocol-aware evaluation that reports the random-split and session-grouped results side by side for the same data and quantifies the inflation caused by window overlap.
- A finding on the interaction between a confidence-based bypass and a stateful stay-down check, with a measured effect on confirmation (from 81.7% to 20.4% of fall sessions in the replay).
- A transparent, template-based explanation and a proactive risk score that need no further learned model.

7.4 Product contribution

The product contribution is a working end-to-end prototype: firmware, broker connection, service, engine, and a live dashboard with eight sections that are part of the AegisCare platform. It uses inexpensive parts (an IMU and a Wi-Fi

microcontroller), a hosted broker, and open-source software, so it can be reproduced and extended by other students. The dashboard turns the technical output into a posture level, an explanation, and a risk score that a non-expert can read.

7.5 Limitations and threats to validity

The conclusions of this thesis must be read with the following limitations, grouped by the type of validity they affect.

Table 7.1 Limitations and threats to validity

Type	Limitation	Effect on the conclusions
External	One subject, one day, one sensor position; falls simulated by a healthy adult	Accuracy and thresholds may not transfer to older adults, who move differently and fall differently
External	Only two non-fall classes (standing, walking); no sitting, lying, bending, stairs, or other daily activities	The false-alarm benefit of the gates against real confounders is not demonstrated
Internal	Thresholds (30°, 25°, 8.0, 12.0) were set from the same data on which the scenarios were run	The scenario results are optimistic; they should be re-run on new recordings
Internal	Stay-down and stumble continuations are synthetic	Confirmation rates are indicative; the effect of the confidence bypass needs real lyingstill data
Construct	Window overlap of 50%; short sessions excluded; actual sampling at about 8 Hz instead of 10 Hz	Window spans about 2.5 s; model and engine assume 2 s
Construct	Firmware smoothing ($\alpha = 0.1$) removes impact peaks; ±2 g range may saturate in a real impact	An impact criterion cannot be used; very hard falls are not represented
Implementation	Single global Fall Engine; no authentication on the API; credentials in source; broker mismatch between firmware and service; HTTP 500 for a non-JSON request	Not deployable for more than one person or in an open network as it stands
Product	No user study; risk-score weights are not clinically validated; fall alert flag is true already at "possible fall"	Acceptability and the meaning of the risk score are unproven

Source: Author.

7.6 Implications

For the AegisCare platform, the main implication is that the alert component should treat "possible fall" and "confirmed fall" as separate events and should use only the confirmed state to notify a caregiver, with the possible state shown as a warning. For fall-detection research, the implication is that a reported accuracy should always be accompanied by the splitting protocol, and that stateful logic around a classifier must

be tested as a whole, with the classifier's own uncertainty rule in place. For caregivers, the implication is that a decision with a visible reason can be checked and trusted or rejected, which is more useful than a bare alarm.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

The component offers a caregiver a low-cost wearable that detects a fall without any action by the wearer, confirms it before alerting, and says why. The primary users are family caregivers of older adults who live alone, and the secondary users are small care homes and community-care programs that need a simple monitoring tool without installing cameras. Three benefits follow from the design: fewer alerts that must be dismissed because non-fall movements are not confirmed as falls (Section 6.2), a decision that can be checked because it is accompanied by measured evidence, and a bill of materials that consists of an inexpensive IMU and a Wi-Fi microcontroller.

8.2 Market and competitive position

Competing solutions are push-button pendants, smartwatch fall-detection features, camera systems, and floor sensors (Table 2.1). The component is positioned between the pendant and the smartwatch: it needs no action like a smartwatch but is built for a single purpose, can be reconfigured, and exposes its reasoning. Its weakness against established products is the lack of clinical evidence, certification, and mobile connectivity.

8.3 Business or sustainability model

Three models are plausible for a later stage: a device sold with a subscription for the dashboard and alerts, a license of the software layer to device makers, and a service contract with care providers. Which of these is viable depends on costs and willingness to pay that have not been investigated here. A sustainability-oriented option is to release the firmware, the training pipeline, and the decision logic as open source for community and academic use, and to charge only for hosted services and support.

8.4 Scalability intellectual property and compliance

Scalability. The service holds one Fall Engine instance, so it supports one wearer. To scale, the engine state must be kept per device, keyed by the MQTT topic or the device identifier, the broker must use one topic per device, and the API needs authentication. Inference with a Random Forest on 52 features is light enough that a single small server can serve many wearers. **Intellectual property.** The code and the recorded data set are

the work of the research group, and the ownership terms are set by the university policy. Libraries used are open source, and their licenses must be checked before distribution. **Compliance.** A product that claims to detect falls for health purposes may be regulated as a medical device in some jurisdictions; the prototype makes no such claim. Personal data would be subject to the Personal Data Protection Act of Sri Lanka [19], which requires consent, minimal collection, and security measures that the prototype does not yet implement (Section 7.5).

8.5 Contribution to the Sustainable Development Goals

The component contributes mainly to Sustainable Development Goal 3, good health and well-being, which includes reducing premature mortality and promoting health at all ages [22]. Faster detection of falls and a record of how often an older person is unstable can support earlier help and earlier prevention. It also supports Goal 9, industry, innovation and infrastructure, through a low-cost IoT design that can be built with local skills, and Goal 10, reduced inequalities, to the extent that an inexpensive device makes monitoring available to families that cannot afford a commercial service. The contribution is potential and not demonstrated: no deployment or outcome data exist.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis presented the design, implementation, and evaluation of the Context-Aware Activity Recognition component of AegisCare: a wearable IMU node, an MQTT transport, a Random Forest classifier, a physics-based Fall Engine that confirms a fall only when the classifier, the body tilt, and a stay-down check agree, a fall-risk score with template explanations, and a live dashboard. With whole recordings held out, the classifier reached 98.15% accuracy, against 100% under the random split used by the training script, which shows how much the evaluation protocol matters. In scripted replays, no standing or walking session was confirmed or flagged as a fall, and recovered stumbles were recognized as recoveries in most sessions.

The work also exposed weaknesses that are as informative as the successes: about one in nine recorded falls never reaches the 30° gate, the deployed confidence rule weakens the stay-down check for motionless postures, the evidence comes from one subject

with simulated falls, and no user study has been carried out. The component is a credible research prototype and a sound basis for further work, but it is not ready for deployment with older adults.

9.2 Achievement of objectives

Table 9.1 Achievement of the objectives

Objective	**Status**	**Evidence**
O1 Sensing node streaming at about 10 Hz	Achieved; actual rate about 8 Hz	Section 3.2.2; firmware
O2 Dataset and three classifiers	Achieved (361 sessions, RF, DT, KNN)	Table 3.2, Table 6.2
O3 Evaluation with whole recordings held out	Achieved	Table 6.2, Table 6.3
O4 Three-gate confirmation engine	Achieved; weakened by the confidence bypass in the deployed service	Table 6.5, Table 6.6
O5 Risk score and explanation	Achieved; score weights not validated	Section 4.4.4; Section 4.4.5
O6 Integration and scripted tests	Achieved for the software; hardware log to be attached	Table 5.2

Source: Author.

9.3 Individual research and product contributions

The research contributions are the three-gate confirmation design, the protocol-aware evaluation, and the finding on the interaction between the confidence rule and the stay -down check (Section 7.3). The product contributions are the working end-to-end prototype and the eight-section dashboard that is part of AegisCare (Section 7.4).

9.4 Future directions

1. **Fix the confirmation logic.** Pass every window to the Fall Engine regardless of confidence, or add a "lying still" class, and repeat the scenarios with recordings of people lying still.
2. **Collect more and better data.** Add subjects, ideally including older adults under an approved protocol, and add confounding activities (sitting, lying down, bending, stairs, picking up objects), which are needed to show the benefit of the gates.

3. **Recalibrate the sensor settings.** Reduce the smoothing or add an unsmoothed channel to recover impact peaks, evaluate the ±4 g range, and fix the sampling rate to the intended 10 Hz.
4. **Learn thresholds with nested validation.** Choose the tilt and stillness thresholds inside the cross-validation loop and report them with confidence intervals.
5. **Validate with users.** Run the think-aloud study proposed in Section 6.3 with caregivers and measure alert burden in a field trial.
6. **Harden the system.** Add authentication, per-device state, secret management, a single broker configuration, an automated regression suite, and a correct HTTP 400 for malformed requests.
7. **Validate the risk score.** Compare it with a recognised fall-risk instrument and learn its weights from graded-risk data.

REFERENCES

[1] World Health Organization, "Falls," *WHO Fact Sheets*, Apr. 26, 2021. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/falls

[2] M. E. Tinetti, M. Speechley, and S. F. Ginter, "Risk factors for falls among elderly persons living in the community," *N. Engl. J. Med.*, vol. 319, no. 26, pp. 1701–1707, 1988.

[3] OASIS, "MQTT Version 3.1.1," OASIS Standard, Oct. 29, 2014. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html

[4] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[5] D. R. Roberts *et al.*, "Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure," *Ecography*, vol. 40, no. 8, pp. 913– 929, 2017.

[6] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, art. no. 100804, 2023.

[7] A. B. Arrieta *et al.*, "Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," *Inf. Fusion*, vol. 58, pp. 82–115, 2020.

[8] Bosch Sensortec, "BMI160: Small, low power inertial measurement unit," Data sheet, Rev. 1.0, 2015.

[9] A. Bulling, U. Blanke, and B. Schiele, "A tutorial on human activity recognition using body-worn inertial sensors," *ACM Comput. Surv.*, vol. 46, no. 3, art. no. 33, 2014.

[10] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[11] N. Noury *et al.*, "Fall detection – principles and methods," in *Proc. 29th Annu. Int. Conf. IEEE Eng. Med. Biol. Soc. (EMBC)*, Lyon, France, 2007, pp. 1663–1666.

[12] M. Mubashir, L. Shao, and L. Seed, "A survey on fall detection: Principles and approaches," *Neurocomputing*, vol. 100, pp. 144–152, 2013.

[13] R. Igual, C. Medrano, and I. Plaza, "Challenges, issues and trends in fall detection systems," *BioMed. Eng. OnLine*, vol. 12, art. no. 66, 2013.

[14] A. K. Bourke, J. V. O'Brien, and G. M. Lyons, "Evaluation of a thresholdbased tri-axial accelerometer fall detection algorithm," *Gait Posture*, vol. 26, no. 2, pp. 194–199, 2007.

[15] M. Kangas, A. Konttila, P. Lindgren, I. Winblad, and T. Jämsä, "Comparison of low-complexity fall detection algorithms for body attached accelerometers," *Gait Posture*, vol. 28, no. 2, pp. 285–291, 2008.

[16] A. T. Özdemir and B. Barshan, "Detecting falls with wearable sensors using machine learning techniques," *Sensors*, vol. 14, no. 6, pp. 10691–10708, 2014.

[17] J. Wang, Y. Chen, S. Hao, X. Peng, and L. Hu, "Deep learning for sensor-based activity recognition: A survey," *Pattern Recognit. Lett.*, vol. 119, pp. 3–11, 2019.

[18] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, "A public domain dataset for human activity recognition using smartphones," in *Proc. 21st Eur. Symp. Artif. Neural Netw. (ESANN)*, Bruges, Belgium, 2013, pp. 437– 442.

[19] Parliament of the Democratic Socialist Republic of Sri Lanka, "Personal Data Protection Act, No. 9 of 2022," Colombo, Sri Lanka, 2022.

[20] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.

[21] ISO/IEC, *Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – System and Software Quality Models*, ISO/IEC 25010:2011, 2011.

[22] United Nations, "Transforming our world: The 2030 Agenda for Sustainable Development," A/RES/70/1, 2015.

GLOSSARY

Accelerometer – Sensor that measures specific force (gravity plus body acceleration) along three axes.

Confidence – The highest class probability returned by the classifier for a window; below 0.60 the window is labelled "uncertain".

Fall Engine – The stateful module in the service that confirms or rejects a fall using the classifier output, tilt, and stillness.

Gate – A condition that must hold before a fall is confirmed; the component has three (classifier, orientation, stay-down).

Gyroscope – Sensor that measures angular velocity about three axes.

Out-of-fold prediction – A prediction for a sample made by a model that did not see that sample's session during training.

Risk score – Heuristic value from 0 to 100 describing how unstable the current movement looks.

Session – One continuous recording of one motion class; the unit that is kept whole in grouped cross-validation.

Stay-down check – Check that the body remains tilted and motionless after a suspected fall.

Tilt – Angle between the sensor's vertical axis and the direction of gravity; 0° when upright.

Window – A block of 20 consecutive samples (about 2 to 2.5 s) from which features are computed.

APPENDICES

Appendix A contains extended results and test evidence, and Appendix B contains the disclosure of the use of AI tools.

Appendix A Extended results and test evidence

This appendix gives the confusion matrix of Figure 6.2 as counts, the API test log of Section 5.4, and the definitions of the scenarios.

Table A.1 Confusion matrix of the Random Forest (rows: actual; columns: predicted; out-offold windows)

Actual \ Predicted	Back.	Fwd.	Left	Walk	Right	Stand.
Backward fall	116	1	0	0	0	4
Forward fall	0	119	0	0	0	0
Left fall	0	0	116	0	0	0
Normal walk	1	0	0	127	0	0
Right fall	0	0	0	1	140	0
Standing still	0	0	0	6	0	71

Source: Author's evaluation.

Table A.2 API test log (Flask test client, MQTT stubbed)

Request	Status	Result
GET /health	200	status ok; six classes; window_size 20
GET /latest (no stream)	200	empty object
POST /reset	200	{"status": "reset"}
POST /predict, real standing window	200	standing_still, confidence 0.995, event normal
POST /predict, real walking window	200	normal_walk, confidence 0.965
POST /predict, random noise window	200	uncertain, confidence 0.3
POST /predict, 5-sample window	400	shape error
POST /predict, empty JSON object	400	Missing window
POST /predict, no body	500	Defect: expected 400
POST /predict, forward-fall session, window 1	200	possible_fall; tilt 40° forward; risk 84.3; fall_alert true
POST /predict, forward-fall session, window 2	200	confirming; risk 95.3; fall_alert true

Source: Author's test script.

Scenario definitions

- **Stay-down:** all windows of a fall session (step 10), then four windows that repeat the last sample with the gyroscope set to zero.

- **Stumble:** all windows of a fall session, then five windows with the sensor upright (ay = 0.99 g) and no motion.
- **Non-fall:** all windows of a standing or walking session, then four repetitions of the last window.
- **Dashboard replay:** samples are fed to the posture engine at 100 ms intervals, followed by 40 samples (4 s) of either a still last posture or an upright posture; in fall scenarios the classifier evidence is the true fall label with confidence 0.9.

Appendix B Academic integrity and AI use disclosure

In accordance with the module policy, the use of generative AI is disclosed in Table B.1.

Table B.1 Use of generative AI tools

Tool	Purpose	Verification by the author	Parts affected
Claude (Anthropic)	Explaining the source code of the component; drafting the structure and text of this thesis from the author's code and results; writing analysis and test scripts	Scripts were run and their output compared with the code and data; numbers in Chapters 3 to 7 come from these runs; literature references to be checked against the original sources	Chapters 1–9, Appendix A; analysis scripts; Figures 6.1–6.4

Source: Author.